PUC Minas



Algoritmos e Estruturas de Dados I

Prof. Dr. João Paulo Aramuni

Aula 05

Abstrações de dados heterogêneos

AEDS I - Manhã

05

Abstrações de dados heterogêneos

Unidade de Ensino

5.1 Noções sobre estruturas

5.2 Noções sobre classes e orientações ao uso de objetos

5.2.1 Definições e acessos

5.2.2 Encapsulamento e herança

5.2.3 Tratamento de exceções

5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária



5.1 Noções sobre estruturas

Quando precisamos construir um sistema personalizado para algum cliente, devemos ter uma ferramenta que seja flexível para as diversas regras de negócio do projeto.

Algo que nos dê liberdade de criar uma estrutura que todas as pessoas desenvolvedoras consigam entender.



5.1 Noções sobre estruturas

Dessa forma, foi criado, com esses objetivos, a estrutura de dados **Struct**, que tem o papel de facilitar a organização de diversas informações.

Na Struct, conseguimos agrupar diversos tipos de dados, portanto, é usado com frequência para registros de tipos de dados **mistos**, algo como um cadastro que requer vários dados diferentes (nome, endereço, telefone, etc.).



5.1 Noções sobre estruturas

As structs em C são usadas para criar abstrações de dados heterogêneos.

Em uma struct (estrutura), você pode agrupar variáveis de diferentes tipos sob um único nome. Cada variável dentro da struct é chamada de membro, e esses membros podem ter tipos de dados diferentes. Isso permite representar dados compostos por diversos tipos de informações relacionadas.



5.1 Noções sobre estruturas

Struct é uma abreviação para Structure, que significa estrutura.

Na linguagem C (e em muitas outras) é uma estrutura de dados composta que define fisicamente uma lista de variáveis agrupadas sob um nome em um bloco de memória, logo, todas as variáveis conseguem ser acessadas por meio de um único ponteiro ou pelo que é declarado na estrutura que retorna o mesmo endereço.



5.1 Noções sobre estruturas

A Struct em C faz a referência direta ao bloco contíguo de memória física, geralmente delimitado (dimensionado) por limites de comprimento de palavra.

A palavra Struct corresponde ao recurso com nome semelhante disponível em algumas linguagens de baixo nível e até processadores Intel. Sendo um bloco de memória contígua, cada campo dentro da estrutura está localizado em um determinado deslocamento fixo desde o início.



5.1 Noções sobre estruturas

Como já comentado, Struct nada mais é do que uma estrutura com diversos tipos de dados, também é conhecido pelo nome de registro. A ideia do Struct é de armazenar os dados em uma mesma entidade, em apenas uma única variável.

Se quisermos guardar o nome, telefone e o e-mail de uma pessoa, podemos criar uma Struct chamada **Pessoa**, por exemplo, e agrupar essas informações em um único tipo de dado.



5.1 Noções sobre estruturas

Definindo uma struct:

Vamos começar criando uma estrutura do zero, e para isso devemos utilizar a instrução Struct para criar um novo tipo de dados. A sintaxe dessa instrução é a seguinte:

struct [tag da estrutura] {
tipo_da_variavel nome_da_variavel;
tipo_da_variável nome_da_variavel;
tipo_da_variavel nome_da_variavel;
} [uma ou mais variáveis da estrutura];

5.1 Noções sobre estruturas

A tag de estrutura é opcional e cada definição de uma variável é igual, como int i; ou float f; ou qualquer outra definição de variável válida.

No final da definição da estrutura, antes do ponto e vírgula final, você pode especificar uma ou mais variáveis de estrutura, mas também é opcional (declarar as variáveis na própria estrutura não é muito recomendável).

5.1 Noções sobre estruturas

Esta é uma maneira de como você declararia uma estrutura de um livro:

```
struct Livro {
char  titulo[50];
char  autor[50];
char  assunto[100];
int   id_livro;
} livro;
```

A declaração das variáveis em uma struct pode ser feita de algumas maneiras, tais como:

5.1 Noções sobre estruturas

Exemplo 1:

```
struct Livro {
char  titulo[50];
char  autor[50];
char  assunto[100];
int   id_livro;
};

struct Livro Livro1, Livro2;

//Declarando Livro1 e Livro2 do tipo Livro
```

5.1 Noções sobre estruturas

Exemplo 2:

```
struct Livro {
char  titulo[50];
char  autor[50];
char  assunto[100];
int   id_livro;
} Livro1, Livro2; // declarando as variáveis na própria estrutura: não é muito recomendável
```

Nesse último, as variáveis Livro1 e Livro2 são criadas a partir da estrutura

Livro, no entanto, essa fórmula não é muito recomendada.

5.1 Noções sobre estruturas

Vejamos agora como inicializar uma struct. Podemos adicionar os valores para os dados de uma struct de duas formas:

```
struct Livro {
char  titulo[50];
char  autor[50];
char  assunto[100];
int   id_livro;
};

struct Livro Livro1 = { "Título genérico", "Autor", "Um livro genérico", 123 };
// inicializando
```

...

5.1 Noções sobre estruturas

Ou também:

```
struct Livro {
char  titulo[50];
char  autor[50];
char  assunto[100];
int   id_livro; };

struct Livro Livro1; //Declarando Livro1 do tipo Livro
strcpy( Livro1.titulo, "Título genérico"); // inicializando os valores para cada variável
strcpy( Livro1.autor, "Autor");
strcpy( Livro1.assunto, "Um livro genérico");
Livro1.id_livro = 123;
```

...

5.1 Noções sobre estruturas

Para acessar algum dado de uma struct, utilizamos o operador de acesso ponto (.).

O operador de acesso é escrito entre o nome da variável e o dado da estrutura que desejamos acessar.

Você também pode usar a palavra-chave struct para declarar variáveis a partir de outras structs. Vejamos um exemplo completo em C.

...

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Livro {
    char titulo[50];
    char autor[50];
    char assunto[100];
    int id_livro;
};

int main(void) {

    struct Livro Livro1; // Declaramos Livro1 do tipo Livro
    struct Livro Livro2; // Declaramos Livro2 do tipo Livro

    // especificações do livro 1
    strcpy(Livro1.titulo, "Título genérico");
    strcpy(Livro1.autor, "Autor");
    strcpy(Livro1.assunto, "Um livro genérico");
    Livro1.id_livro = 123;

    // especificações do livro 2
    strcpy(Livro2.titulo, "Outro título genérico");
    strcpy(Livro2.autor, "Autor 2");
    strcpy(Livro2.assunto, "Outro livro genérico");
    Livro2.id_livro = 456;

    // mostrando as informações do livro 1
    printf("Livro 1 titulo : %s\n", Livro1.titulo);
    printf("Livro 1 autor : %s\n", Livro1.autor);
    printf("Livro 1 assunto : %s\n", Livro1.assunto);
    printf("Livro 1 id_livro : %d\n", Livro1.id_livro);

    // mostrando as informações do livro 2
    printf("Livro 2 titulo : %s\n", Livro2.titulo);
    printf("Livro 2 autor : %s\n", Livro2.autor);
    printf("Livro 2 assunto : %s\n", Livro2.assunto);
    printf("Livro 2 id_livro : %d\n", Livro2.id_livro);

    return 0;
}
```

Console × Problems Properties

<terminated> (exit value: 0) Prj_Struct [C/C++ Application]

```
Livro 1 titulo : Título genérico
Livro 1 autor : Autor
Livro 1 assunto : Um livro genérico
Livro 1 id_livro : 123
Livro 2 titulo : Outro título genérico
Livro 2 autor : Autor 2
Livro 2 assunto : Outro livro genérico
Livro 2 id_livro : 456
```

5.1 Noções sobre estruturas

Você pode passar uma struct como um argumento de uma função da mesma maneira que passa qualquer outra variável ou ponteiro, vejamos um exemplo:

```
void printLivro(struct Livro livro) {
printf( "Livro titulo : %s\n", livro.titulo);
printf( "Livro autor : %s\n", livro.autor);
printf( "Livro assunto : %s\n", livro.assunto);
printf( "Livro id_livro : %d\n", livro.id_livro);
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Livro {
    char titulo[50];
    char autor[50];
    char assunto[100];
    int id_livro;
};

// Protótipo da função (assinatura)
void printLivro(struct Livro livro);

int main(void) {

    struct Livro Livro1; // Declaramos Livro1 do tipo Livro
    struct Livro Livro2; // Declaramos Livro2 do tipo Livro

    // especificações do livro 1
    strcpy(Livro1.titulo, "Título genérico");
    strcpy(Livro1.autor, "Autor");
    strcpy(Livro1.assunto, "Um livro genérico");
    Livro1.id_livro = 123;

    // especificações do livro 2
    strcpy(Livro2.titulo, "Outro título genérico");
    strcpy(Livro2.autor, "Autor 2");
    strcpy(Livro2.assunto, "Outro livro genérico");
    Livro2.id_livro = 456;

    // mostrando as informações do livro 1
    printLivro(Livro1);

    // mostrando as informações do livro 1
    printLivro(Livro2);

    return 0;
}
```

```c
void printLivro(struct Livro livro) {
    printf("Livro titulo : %s\n", livro.titulo);
    printf("Livro autor : %s\n", livro.autor);
    printf("Livro assunto : %s\n", livro.assunto);
    printf("Livro id_livro : %d\n", livro.id_livro);
}
```

Console | Problems | Properties | Search | Tasks

```
<terminated> (exit value: 0) Prj_Struct_Arg [C/C++ Application] /Users/joaopauloaramuni/Docume
Livro titulo : Título genérico
Livro autor : Autor
Livro assunto : Um livro genérico
Livro id_livro : 123
Livro titulo : Outro título genérico
Livro autor : Autor 2
Livro assunto : Outro livro genérico
Livro id_livro : 456
```

5.1 Noções sobre estruturas

Você pode definir ponteiros para structs da mesma forma que define ponteiros para qualquer outra variável.

struct Livro *struct_ponteiro;

Dessa maneira, você consegue armazenar o endereço de uma variável de struct em uma variável de ponteiro definida acima.

5.1 Noções sobre estruturas

Para encontrar o endereço de uma variável de estrutura, coloque o operador **&** antes do nome da estrutura da seguinte forma:

struct_ponteiro = **&**Livro1;

Para acessarmos os dados de uma estrutura usando um ponteiro, devemos utilizar o operador seta **->** da seguinte maneira:

struct_ponteiro**->**titulo;

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Livro {
    char titulo[50];
    char autor[50];
    char assunto[100];
    int id_livro;
};

// Protótipo da função (assinatura)
void printLivro(struct Livro *livro);

int main(void) {

    struct Livro Livro1; // Declarando Livro1 do tipo Livro
    struct Livro Livro2; // Declarando Livro2 do tipo Livro

    // especificações do livro 1
    strcpy(Livro1.titulo, "Título genérico");
    strcpy(Livro1.autor, "Autor");
    strcpy(Livro1.assunto, "Um livro genérico");
    Livro1.id_livro = 123;

    // especificações do livro 2
    strcpy(Livro2.titulo, "Outro título genérico");
    strcpy(Livro2.autor, "Autor 2");
    strcpy(Livro2.assunto, "Outro livro genérico");
    Livro2.id_livro = 456;

    // mostrando informações do Livro1 passando o endereço da memória
    printLivro(&Livro1);

    // mostrando informações do Livro2 passando o endereço da memória
    printLivro(&Livro2);

    return 0;
}
```

```c
void printLivro(struct Livro *livro) {
    printf("Livro titulo : %s\n", livro->titulo);
    printf("Livro autor : %s\n", livro->autor);
    printf("Livro assunto : %s\n", livro->assunto);
    printf("Livro id_livro : %d\n", livro->id_livro);
}
```

Console | Problems | Properties | Search | Tasks

<terminated> (exit value: 0) Prj_Struct_Ponteiro [C/C++ Application] /Users/joaopauloaramuni/Docu

```
Livro titulo : Título genérico
Livro autor : Autor
Livro assunto : Um livro genérico
Livro id_livro : 123
Livro titulo : Outro título genérico
Livro autor : Autor 2
Livro assunto : Outro livro genérico
Livro id_livro : 456
```

5.1 Noções sobre estruturas

O principal objetivo da Struct, como já vimos, é criar uma estrutura de dados que junte vários tipos de dados em apenas um bloco de memória, tanto para facilitar o acesso, como também melhorar o entendimento da arquitetura do programa.

Podemos utilizar a Struct para criar diversas estruturas personalizadas para o nosso sistema.

5.1 Noções sobre estruturas

Por exemplo, se quisermos criar um sistema que cadastre uma **pessoa**, é recomendável criar uma Struct com todas as informações necessárias para esse cadastro, algo como, nome, endereço, idade, altura...

Com essas informações, criamos uma Struct declarando os tipos das variáveis e os seus respectivos nomes. Logo após isso, basta utilizar a estrutura em qualquer local do código.

5.1 Noções sobre estruturas

Seguindo nessa linha, temos a chance de criar funções que aceitam o parâmetro personalizado da nossa estrutura de dados e que pode ou não retornar também uma outra estrutura de dados, conseguimos até mesmo modificar os valores através dos operadores de acesso que são disponibilizados pela linguagem.

Por fim, podemos reutilizar todas essas informações em várias partes do nosso código e torná-lo muito mais simples.

5.1 Noções sobre estruturas

O **typedef** é uma palavra-chave em C que permite criar alias (apelidos) para tipos de dados existentes e pode ser muito útil em structs.

Isso é adequado para tornar o código mais legível, facilitar a manutenção e fornecer um meio de criar tipos personalizados.

A sintaxe básica do typedef é a seguinte:

5.1 Noções sobre estruturas

```c
#include <stdio.h>
#include <stdlib.h>

// Criando um alias para o tipo 'int' no escopo global
typedef int MeuInteiro;

int main(void) {
    // Usando o novo tipo criado com typedef
    MeuInteiro numero = 13;

    printf("Número: %d\n", numero);

    return 0;
}
```

5.1 Noções sobre estruturas

A principal vantagem do **typedef** em structs está na capacidade de criar um nome mais curto e descritivo para um tipo de dados, eliminando a necessidade de usar a palavra-chave struct ao declarar variáveis desse tipo.

Isso melhora a legibilidade do código e pode facilitar alterações futuras no tipo subjacente. Vejamos mais um exemplo de struct, agora com typedef.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { // Cria uma struct para armazenar os dados de uma pessoa
    float Peso;   // define o campo Peso
    int Idade;    // define o campo Idade
    float Altura; // define o campo Altura
} Pessoa; // Define o nome do novo tipo criado

void ImprimePessoa(Pessoa P){ // declara o parâmetro como uma struct
    printf("Idade: %d  Peso: %.2f Altura: %.2f\n", P.Idade, P.Peso, P.Altura);
}

int main(void) {
    Pessoa Joao, P2;
    Pessoa Povo[10];

    Joao.Idade = 15;
    Joao.Peso = 60.5;
    Joao.Altura = 1.75;

    Povo[4].Idade = 23;
    Povo[4].Peso = 75.3;
    Povo[4].Altura = 1.89;

    /*Outra vantagem de utilizar struct é a possibilidade de atribuir os dados de uma
    struct para outra, com apenas um comando de atribuição, como neste exemplo: */
    P2 = Povo[4];
    P2.Idade++;

    // chama a função que recebe a struct como parâmetro
    ImprimePessoa(Joao);
    ImprimePessoa(Povo[4]);
    ImprimePessoa(P2);
    return 0;
}
```

Console | Problems | Properties

<terminated> (exit value: 0) Prj_Struct_Pessoa [C/C++

```
Idade: 15  Peso: 60.50 Altura: 1.75
Idade: 23  Peso: 75.30 Altura: 1.89
Idade: 24  Peso: 75.30 Altura: 1.89
```

Passagem de structs por parâmetros

5.1 Noções sobre estruturas

Como uma struct define um tipo de dado, este tipo pode ser retornado em uma função, da mesma forma que ocorre com qualquer outro tipo de dado.

No exemplo a seguir, a função retorna uma struct que conterá, em seus campos, os dados que foram recebidos por parâmetro.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { // Cria uma struct para armazenar os dados de uma pessoa
    float Peso;   // define o campo Peso
    int Idade;    // define o campo Idade
    float Altura; // define o campo Altura
} Pessoa; // Define o nome do novo tipo criado

Pessoa SetPessoa(int idade, float peso, float altura) {
    Pessoa P;
    P.Idade = idade;
    P.Peso = peso;
    P.Altura = altura;
    return P; // retorna a struct contendo os dados passados por parâmetro
}

void ImprimePessoa(Pessoa P) { // declara o parâmetro como uma struct
    printf("Idade: %d  Peso: %.2f Altura: %.2f\n", P.Idade, P.Peso, P.Altura);
}

int main(void) {
    Pessoa Joao;
    Joao = SetPessoa(15, 60.5, 1.75); // atribui o retorno da função a uma variável struct
    ImprimePessoa(Joao);
    return 0;
}
```

```
Console  Problems  Properties
<terminated> (exit value: 0) Prj_Struct_Pessoa_Return
Idade: 15  Peso: 60.50 Altura: 1.75
```

Passagem de structs em funções

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { // Cria uma struct para armazenar os dados de uma pessoa
    float Peso;   // define o campo Peso
    int Idade;    // define o campo Idade
    float Altura; // define o campo Altura
} Pessoa; // Define o nome do novo tipo criado

void ImprimePessoa(Pessoa P) {
    printf("Idade: %d  Peso: %.2f Altura: %.2f\n", P.Idade, P.Peso, P.Altura);
}

// Nesta função o parâmetro P é um ponteiro para uma struct
void SetPessoa(Pessoa *P, int idade, float peso, float altura) {
    (*P).Idade = idade;  // o campo pode ser acessado desta forma
    P->Peso = peso;      // ou desta
    P->Altura = altura;
}

int main(void) {
    Pessoa Joao;
    SetPessoa(&Joao, 15, 70.5, 1.75);
    ImprimePessoa(Joao);
    return 0;
}
```

Console × Problems Properties

<terminated> (exit value: 0) Prj_Struct_Pessoa_Ponteiro

```
Idade: 15  Peso: 70.50 Altura: 1.75
```

Passagem de structs por referência

5.1 Noções sobre estruturas

A principal vantagem de uma Struct é que temos a possibilidade de armazenar diversos tipos de dados que não são possíveis em arrays. Os arrays só podem lidar com o mesmo tipo de dado.

Recorrendo a algumas práticas incertas, conseguimos guardar e acessar as informações em uma array, porém acabamos perdendo a identidade particular que relaciona os dados a apenas um lugar.

5.1 Noções sobre estruturas

A principal diferença entre um **Array** e uma **Struct** é que um Array suporta vários dados de apenas um tipo e uma Struct consegue guardar vários dados de diversos tipos.

Podemos também observar algumas outras diferenças entre essas estruturas de dados. Vejamos o quadro a seguir.

5.1 Noções sobre estruturas

Base para comparação	Array	Struct
Conceito básico	Um array é uma coleção de variáveis do mesmo tipo de dados.	Uma struct é uma coleção de variáveis de diferentes tipos de dados.
Memória	Os elementos de um array são armazenados em um local de memória contíguo.	Os elementos da struct não podem ser armazenados em um local de memória contíguo (padding) e o tamanho de cada elemento, bem como o espaço reservado para ele, variará de acordo com seu tipo.
Acesso	Os elementos do array são acessados por seu número de índice.	Os elementos da struct são acessados por seus nomes.
Ponteiro	O nome do array aponta para o primeiro elemento desse array, portanto, o nome da array é um ponteiro.	O nome da struct não aponta para o primeiro elemento dessa struct, portanto, o nome da struct não é um ponteiro.
Tamanho	Cada elemento do array tem o mesmo tamanho.	Cada elemento em uma struct possui diferentes tipos de dados.
Tempo de acesso	Acessar o elemento do array requer menos tempo.	Acessar os elementos de uma struct requer comparativamente mais tempo.
Busca	Pesquisar um elemento do array leva menos tempo.	Pesquisar um elemento de uma struct leva comparativamente mais tempo do que um elemento da array.

5.1 Noções sobre estruturas

Os array são acessadas rapidamente, já que o tamanho de cada elemento em um array é constante.

Já na struct, temos um acesso um pouco mais defasado, já que cada elemento de uma variável da estrutura tem um tamanho diferente entre si. Isso pode resultar em um espaçamento entre os elementos (chamado de "padding") para alinhamento na memória. Este alinhamento pode resultar em espaços vazios entre os elementos.

5.1 Noções sobre estruturas

Utilizando struct conseguimos criar uma estrutura de dados para guardar e acessar todos os dados de um tipo sem perder a sua identidade particular, mesmo tendo uma enorme quantidade de dados.

Somos capazes também de simplificar o nosso código, de uma maneira que, se for necessário passar vários parâmetros para uma função, conseguimos criar uma estrutura que pode ser utilizada para isso.

5.1 Noções sobre estruturas

Vejamos algumas limitações na forma de criar e utilizar a struct em C:

- Não é permitido que uma estrutura seja tratada como um tipo de dados integrado.
- Não conseguimos usar os operadores matemáticos, como o +, -, etc.
- As estruturas em C não suportam a ocultação de dados. Os membros da estrutura conseguem ser acessados por qualquer função, em qualquer lugar no escopo da struct.

5.1 Noções sobre estruturas

Vejamos algumas limitações na forma de criar e utilizar a struct em C:

- Não é permitido adicionar funções dentro de uma struct.
- Também não é possível adicionar dados estáticos dentro de seu corpo.
- Na linguagem C não temos o suporte aos modificadores de acesso (público, privado, etc.), logo, eles não podem ser usados nas structs.
- Por último, não se pode colocar construtores dentro de uma struct.

...

5.1 Noções sobre estruturas

Outra estrutura de dados muito interessante em C é a do tipo **union**.

As estruturas de dados do tipo Union permite armazenar diferentes tipos de dados no mesmo local de memória. A grande vantagem dessa estrutura esta na organização da memória, e no seu reaproveitamento, isto é, as unions fornecem uma maneira eficiente de usar o mesmo local de memória para vários propósitos.

5.1 Noções sobre estruturas

Ao contrário de uma struct, onde cada membro tem seu próprio espaço na memória, em uma union, todos os membros compartilham o mesmo espaço.

A principal característica de uma union é que ela economiza espaço, pois apenas um dos membros pode ser usado por vez. Todos os membros começam no mesmo endereço de memória, e o tamanho total da union é determinado pelo maior membro.

5.1 Noções sobre estruturas

Para definir uma union, você deve usar a instrução union da mesma forma que definiu uma struct.

A instrução union também define um novo tipo de dado com mais de um membro para o seu programa.

Vejamos o formato da declaração da **union**.

5.1 Noções sobre estruturas

Definição da Union:

union [Nome_do_Tipo_Union] {
Tipo_de_dado1 variavel1;
Tipo_de_dado2 variavel2;
Tipo_de_dadoN variavelN;
} [uma ou mais variaveis Union];

*Nome_do_Tipo_Union é opcional. Não é necessário definir.

Vejamos um exemplo de uso da Union.

5.1 Noções sobre estruturas

No exemplo a seguir, a variável dado é uma union e poderá ser do tipo int, float ou cadeia de caracteres. Depende do valor que será movido.

```
union {
    int i;
    float f;
    char str[20];
} dado;
```

5.1 Noções sobre estruturas

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

union {
    int i;
    float f;
    char str[20];
} dado;

int main(void) {
    dado.i = 10; /* union sera do tipo inteiro */
    printf("Sou inteiro : %d\n", dado.i);
    dado.f = 34.5; /* union sera do tipo float */
    printf("Sou real : %f\n", dado.f);
    strcpy(dado.str, "Sou String"); /* union sera do tipo String */
    printf("Sou string : %s\n", dado.str);

    return 0;
}
```

Console × Problems Properties Search Tasks

<terminated> (exit value: 0) Prj_Union [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_Union/Debug/Prj_Union (16/02/2024 14:42)

```
Sou inteiro : 10
Sou real : 34.500000
Sou string : Sou String
```

Podemos, inclusive, utilizar a union dentro de uma struct:

```c
#include <stdio.h>
#include <stdlib.h>

// Definição da union
union DadosAdicionais {
    int valorInteiro;
    float valorFlutuante;
    char texto[20];
};

// Definição da struct com uma union
struct Pessoa {
    char nome[50];
    int idade;
    union DadosAdicionais dadosExtras;
};

int main(void) {
    // Criando uma instância da struct Pessoa
    struct Pessoa pessoa1 = { "João", 25, { .valorInteiro = 123} };
    struct Pessoa pessoa2 = { "Maria", 30, { .valorFlutuante = 42.0f } };
    struct Pessoa pessoa3 = { "Carlos", 22, { .texto = "Texto Extra" } };

    // Exibindo informações das pessoas
    printf("Nome: %s, Idade: %d, Dados Extras: %d\n", pessoa1.nome,
           pessoa1.idade, pessoa1.dadosExtras.valorInteiro);
    printf("Nome: %s, Idade: %d, Dados Extras: %.2f\n", pessoa2.nome,
           pessoa2.idade, pessoa2.dadosExtras.valorFlutuante);
    printf("Nome: %s, Idade: %d, Dados Extras: %s\n", pessoa3.nome,
           pessoa3.idade, pessoa3.dadosExtras.texto);

    return 0;
}
```

Console | Problems | Properties | Search | Tasks

```
<terminated> (exit value: 0) Prj_Union_Struct [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_Union_Struct/
Nome: João, Idade: 25, Dados Extras: 123
Nome: Maria, Idade: 30, Dados Extras: 42.00
Nome: Carlos, Idade: 22, Dados Extras: Texto Extra
```

5.1 Noções sobre estruturas

Agora que exploramos a ideia de union, vamos mudar nosso foco para outra construção importante em C: **enum**, abreviação de enumeração.

Enquanto as unions são úteis quando queremos compartilhar o mesmo espaço de memória para diferentes tipos de dados, os enums são excelentes para criar conjuntos de constantes nomeadas para representar valores simbólicos em nosso programa.

5.1 Noções sobre estruturas

Vamos explorar como um enum pode ser aplicado para fornecer nomes significativos a um conjunto específico de valores.

Isso não apenas melhora a legibilidade do código, mas também facilita a manutenção e compreensão do propósito desses valores no contexto do programa.

5.1 Noções sobre estruturas

As enumerações, são um tipo definido pelo usuário, utilizando-se de uma lista de identificadores.

Os identificadores desta lista, se assemelham a constantes. As enumerations definem um novo tipo de variável e limita desde logo os valores.

Vejamos o formato da declaração de um **enum**.

5.1 Noções sobre estruturas

O formato da declaração da enum é o seguinte:

enum [Nome_do_Tipo_enum] {
Identificador1;
identificador2;
identificadorN;
} [uma ou mais variaveis enum];

Exemplo:

enum {black, blue, green, cyan, red, purple, yellow, white} cores;

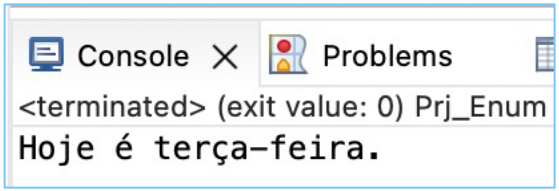
```c
#include <stdio.h>
#include <stdlib.h>

// Definição do enum para dias da semana
enum DiaSemana {
    DOMINGO, SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA, SABADO
};

int main(void) {
    // Declaração de uma variável do tipo enum DiaSemana
    enum DiaSemana hoje = TERCA;

    // Uso do enum em uma estrutura de controle (switch)
    switch (hoje) {
    case DOMINGO:
        printf("Hoje é domingo.\n");
        break;
    case SEGUNDA:
        printf("Hoje é segunda-feira.\n");
        break;
    case TERCA:
        printf("Hoje é terça-feira.\n");
        break;
    case QUARTA:
        printf("Hoje é quarta-feira.\n");
        break;
    case QUINTA:
        printf("Hoje é quinta-feira.\n");
        break;
    case SEXTA:
        printf("Hoje é sexta-feira.\n");
        break;
    case SABADO:
        printf("Hoje é sábado.\n");
        break;
    default:
        printf("Dia desconhecido.\n");
        break;
    }

    return 0;
}
```



Podemos, inclusive, utilizar o enum dentro de uma struct:

```c
#include <stdio.h>
#include <stdlib.h>

// Definição do enum para representar os dias da semana
enum DiaSemana {
    DOMINGO, SEGUNDA, TERCA, QUARTA, QUINTA, SEXTA, SABADO
};

// Definição da struct com um campo do tipo enum
struct Compromisso {
    char descricao[100];
    enum DiaSemana dia;
    int hora;
};

int main(void) {
    // Criando uma instância da struct Compromisso com um valor do enum
    struct Compromisso compromisso1 = { "Reunião de equipe", SEGUNDA, 14 };
    struct Compromisso compromisso2 = { "Almoço com cliente", QUARTA, 12 };
    struct Compromisso compromisso3 = { "Treino na academia", SEXTA, 18 };

    // Exibindo informações dos compromissos
    printf("Compromisso 1: %s, Dia: %d, Hora: %d\n", compromisso1.descricao,
           compromisso1.dia, compromisso1.hora);
    printf("Compromisso 2: %s, Dia: %d, Hora: %d\n", compromisso2.descricao,
           compromisso2.dia, compromisso2.hora);
    printf("Compromisso 3: %s, Dia: %d, Hora: %d\n", compromisso3.descricao,
           compromisso3.dia, compromisso3.hora);

    return 0;
}
```

Console | Problems | Properties | Search | Tasks

<terminated> (exit value: 0) Prj_Enum_Struct [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_Enum_Struct/Debug/Prj_Enum_Struct

```
Compromisso 1: Reunião de equipe, Dia: 1, Hora: 14
Compromisso 2: Almoço com cliente, Dia: 3, Hora: 12
Compromisso 3: Treino na academia, Dia: 5, Hora: 18
```

5.2 Noções sobre classes e orientações ao uso de objetos

Como vimos, em C, structs são utilizadas para criar tipos de dados personalizados que podem armazenar diferentes tipos de variáveis sob um mesmo nome. Elas são frequentemente usadas para agrupar dados relacionados.

Por outro lado, em linguagens de programação orientadas a objetos, como C++, C#, Java e Python, temos o conceito de **classes** e **objetos**.



5.2 Noções sobre classes e orientações ao uso de objetos

Classes:

Em programação orientada a objetos, uma classe é uma estrutura que define um tipo de objeto.

Ela descreve os atributos (dados) e comportamentos/métodos (funções) que os objetos desse tipo possuirão. Uma classe é como um plano ou modelo para criar objetos.



5.2 Noções sobre classes e orientações ao uso de objetos

Objetos:

Um objeto é uma instância de uma classe.

É uma entidade concreta que é criada com base no modelo definido pela classe. Os objetos têm atributos específicos e podem realizar ações específicas, de acordo com os métodos definidos na classe.



5.2 Noções sobre classes e orientações ao uso de objetos

Em C++, em particular, structs e classes são mais semelhantes. Em C++, uma struct pode ter funções membros, acesso a modificadores (public, private, protected), herança e outros recursos que tradicionalmente estão associados a classes.

A principal diferença entre structs e classes em C++ é a visibilidade padrão de membros: em structs, a visibilidade padrão é pública, enquanto em classes é privada.



5.2 Noções sobre classes e orientações ao uso de objetos

Vejamos algumas diferenças entre structs em C e classes em linguagens orientadas a objetos:

Agrupamento de dados:

Structs em C: São usadas para agrupar diferentes tipos de dados sob um mesmo nome.

Classes/Objetos: Em linguagens orientadas a objetos, classes também são usadas para agrupar dados (atributos) e métodos (funções) relacionados.



5.2 Noções sobre classes e orientações ao uso de objetos

Vejamos algumas diferenças entre structs em C e classes em linguagens orientadas a objetos:

Encapsulamento:

Structs em C: Não suportam encapsulamento diretamente. Todos os membros de uma struct são geralmente acessíveis de forma direta.

Classes/Objetos: Encapsulamento é um conceito fundamental. A visibilidade dos membros é controlada (público, privado, protegido) para proteger os dados internos da classe.



5.2 Noções sobre classes e orientações ao uso de objetos

Vejamos algumas diferenças entre structs em C e classes em linguagens orientadas a objetos:

Métodos e funções:

Structs em C: Não podem conter métodos. Normalmente, as funções operam em structs passadas como argumentos.

Classes/Objetos: Além de dados, classes também podem conter métodos que operam nesses dados. Isso permite uma abstração mais completa e facilita a organização do código.



5.2 Noções sobre classes e orientações ao uso de objetos

Vejamos algumas diferenças entre structs em C e classes em linguagens orientadas a objetos:

Herança:

Structs em C: Não suportam herança.

Classes/Objetos: Herança permite que uma classe herde atributos e métodos de outra classe, facilitando a reutilização de código.



5.2 Noções sobre classes e orientações ao uso de objetos

Vejamos algumas diferenças entre structs em C e classes em linguagens orientadas a objetos:

Polimorfismo:

Structs em C: Não suportam polimorfismo diretamente.

Classes/Objetos: Polimorfismo permite que objetos de classes diferentes sejam tratados de maneira uniforme, facilitando a extensibilidade do código.



5.2 Noções sobre classes e orientações ao uso de objetos

Enquanto structs em C fornecem um meio básico de agrupar dados, as classes e objetos em linguagens orientadas a objetos oferecem um conjunto mais rico de recursos, como encapsulamento, herança, polimorfismo, e a capacidade de definir métodos diretamente associados aos dados.



5.2 Noções sobre classes e orientações ao uso de objetos

A Programação Orientada a Objetos (POO) oferece diversos benefícios aos desenvolvedores.

Ela facilita a reutilização de código, proporciona uma organização estruturada e promove a segurança por meio do encapsulamento. A herança e o polimorfismo contribuem para a extensibilidade do código, enquanto a manutenção torna-se mais fácil devido à menor propagação de mudanças.



5.2 Noções sobre classes e orientações ao uso de objetos

Além disso, a POO permite modelar sistemas de software de forma intuitiva, mapeando entidades do mundo real. Ao dominar esses princípios, os desenvolvedores adquirem uma ferramenta poderosa para criar sistemas mais robustos e fáceis de manter.

Por oferecer suporte nativo à orientação por objetos, usaremos C++ nos próximos exemplos de código.



5.2 Noções sobre classes e orientações ao uso de objetos

É importante ressaltar que nosso foco não estará centrado nos detalhes específicos da sintaxe da linguagem C++, mas sim nos conceitos e nos pilares fundamentais da programação orientada a objetos (POO).

A abordagem se concentra em compreender e explorar os princípios universais que fundamentam a POO, tais como encapsulamento, herança, polimorfismo e abstração.



5.2 Noções sobre classes e orientações ao uso de objetos

Ao priorizar esses conceitos, buscamos fornecer uma compreensão sólida e transferível desses fundamentos, independentemente da linguagem de programação utilizada, promovendo uma base conceitual robusta na programação orientada a objetos.

POO é um dos paradigmas mais utilizados na programação e as linguagens que seguem este padrão estão entre as mais utilizadas no mundo segundo pesquisa do stack overflow.



5.2.1 Definições e acessos

Vejamos um exemplo de código em C++ que apresenta a definição e a implementação de uma classe denominada Carro, destinada a representar informações sobre carros.

A classe possui membros públicos, incluindo marca, modelo e ano. Dois construtores são definidos: um construtor padrão que cria um objeto vazio e outro construtor parametrizado que permite a criação de um carro com valores específicos.



5.2.1 Definições e acessos

O método exibirInformacoes é implementado para imprimir detalhes do carro, como marca, modelo e ano.

No interior da classe, são utilizados construtores para inicializar os objetos com valores específicos.

A função principal (main) ilustra a criação de um objeto meuCarro e a atribuição de valores aos seus membros por meio da notação de ponto.



5.2.1 Definições e acessos

Por fim, o método exibirInformacoes é chamado para mostrar na saída padrão as informações do carro.

O código destaca a utilização de classes em C++ para modelar entidades do mundo real. Essa abordagem permite criar e acessar instâncias da classe, definir propriedades específicas para cada carro e utilizar métodos para apresentar as informações de maneira coerente.



5.2.1 Definições e acessos

Vejamos um exemplo de classe implementada em C++:

```cpp
#include <iostream>
#include <string>

// Definição da classe Carro
class Carro {
public:
    // Membros da classe
    std::string marca;
    std::string modelo;
    int ano;

    // Construtor
    Carro(const std::string &m, const std::string &mod, int a) :
            marca(m), modelo(mod), ano(a) {
    }

    // Construtor para criar um carro vazio
    Carro() : marca(""), modelo(""), ano(0) {
    }

    // Método para exibir informações sobre o carro
    void exibirInformacoes() {
        std::cout << "Marca: " << marca << ", Modelo: " << modelo << ", Ano: "
                << ano << std::endl;
    }
};
```

5.2.1 Definições e acessos

Vejamos um exemplo de classe implementada em C++:

```
36 int main(void) {
37     // Criando um objeto da classe Carro
38     Carro meuCarro;
39
40     // Criando um objeto da classe Carro usando o construtor
41     //Carro meuCarro("Toyota", "Corolla", 2022);
42
43     // Atribuindo valores aos membros da classe
44     meuCarro.marca = "Toyota";
45     meuCarro.modelo = "Corolla";
46     meuCarro.ano = 2022;
47
48     // Chamando o método para exibir informações sobre o carro
49     meuCarro.exibirInformacoes();
50
51     return 0;
52 }
```

Console | Problems | Properties | Search | Tasks

<terminated> (exit value: 0) Prj_Classe_Cpp [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_ClasseCarro_

```
Marca: Toyota, Modelo: Corolla, Ano: 2022
```

5.2.1 Definições e acessos

Em C++, um método construtor é uma função especial dentro de uma classe que é automaticamente chamada quando um objeto dessa classe é criado.

Sua principal responsabilidade é realizar as inicializações necessárias para garantir que o objeto esteja em um estado válido e consistente desde o início.

5.2.1 Definições e acessos

Neste trecho de código em C++, é apresentada a implementação de dois métodos construtores para a classe Carro.

```
// Construtor
Carro(const std::string &m, const std::string &mod, int a) :
        marca(m), modelo(mod), ano(a) {
}

// Construtor para criar um carro vazio
Carro() : marca(""), modelo(""), ano(0) {
}
```

5.2.1 Definições e acessos

O primeiro método construtor recebe três parâmetros (uma string para a marca, outra para o modelo e um inteiro para o ano) e utiliza uma lista de inicialização para atribuir esses valores aos membros correspondentes da classe Carro (marca, modelo e ano).

Esse construtor permite a criação de um objeto Carro com valores específicos fornecidos no momento da instanciação.

5.2.1 Definições e acessos

O segundo método construtor é uma versão padrão, conhecida como construtor padrão.

Este construtor não possui parâmetros e inicializa um objeto Carro com valores padrão, neste caso, uma marca vazia, um modelo vazio e um ano igual a zero. Ele é útil quando desejamos criar um objeto Carro sem fornecer valores iniciais, deixando-os em um estado padrão.

5.2.1 Definições e acessos

Dessa forma, a presença de ambos os construtores oferece flexibilidade ao programador, permitindo a criação de objetos Carro com valores específicos ou em um estado padrão, dependendo das necessidades do código.

Esse conceito é conhecido como **sobrecarga** de métodos construtores, pois há mais de uma versão disponível, cada uma com diferentes parâmetros ou nenhum parâmetro.

5.2.2 Encapsulamento e herança

Encapsulamento refere-se à prática de esconder os detalhes internos de uma classe e permitir o acesso controlado a esses detalhes por meio de métodos públicos.

Em C++, isso é alcançado por meio da declaração de membros de classe como privados, protegidos ou públicos. Membros privados são acessíveis apenas dentro da própria classe, protegidos são acessíveis por subclasses, e públicos são acessíveis de fora da classe.



5.2.2 Encapsulamento e herança

O encapsulamento ajuda a proteger a integridade dos dados da classe, fornecendo uma interface controlada para interação externa.

Isso melhora a modularidade, facilita a manutenção e reduz o acoplamento entre diferentes partes do código.



5.2.2 Encapsulamento e herança

Herança é um conceito fundamental em programação orientada a objetos que permite a criação de novas classes baseadas em classes existentes.

Em C++, a herança permite que uma classe (chamada de classe derivada ou subclasse) herde características e comportamentos de outra classe (chamada de classe base ou superclasse).



5.2.2 Encapsulamento e herança

Isso promove a reutilização de código, uma vez que a classe derivada pode acessar membros e métodos da classe base (evita-se duplicar código, facilitando a manutenção do projeto e o trabalho em equipe).

Além disso, a herança facilita a criação de uma hierarquia de classes, organizando conceitos relacionados de forma mais intuitiva.



5.2.2 Encapsulamento e herança

De forma resumida, herança permite a criação de novas classes com base em classes existentes, promovendo a reutilização de código.

Já o encapsulamento controla o acesso aos detalhes internos de uma classe, melhorando a segurança e a manutenção do código. Ambos são princípios essenciais na construção de programas robustos e flexíveis em C++.



5.2.2 Encapsulamento e herança

Vejamos um exemplo de classe que utiliza os mecanismos de encapsulamento e herança em C++: [Classe mãe/base]

```cpp
#include <iostream>
#include <string>

// Classe base: Veiculo
class Veiculo {
private:
    std::string modelo;

protected:
    int ano;

public:
    Veiculo(const std::string &mod, int a) :
            modelo(mod), ano(a) {
    }

    void exibirInformacoes() {
        std::cout << "Modelo: " << modelo << ", Ano: " << ano << std::endl;
    }
};
```

5.2.2 Encapsulamento e herança

Vejamos um exemplo de classe que utiliza os mecanismos de encapsulamento e herança em C++: [Classe filha/derivada]

```
// Classe derivada: Carro (herda de Veiculo)
class Carro: public Veiculo {
private:
    std::string marca;

public:
    Carro(const std::string &m, const std::string &mod, int a) :
            Veiculo(mod, a), marca(m) {
    }

    void exibirDetalhes() {
        std::cout << "Marca: " << marca << ", ";
        exibirInformacoes();  // Chama o método da classe base
    }
};
```

5.2.2 Encapsulamento e herança

Vejamos um exemplo de classe que utiliza os mecanismos de encapsulamento e herança em C++:

```
46 int main(void) {
47     // Criando um objeto da classe Carro
48     Carro meuCarro("Toyota", "Corolla", 2022);
49
50     // Chamando o método para exibir detalhes do carro (que inclui informações do veículo)
51     meuCarro.exibirDetalhes();
52
53     return 0;
54 }
```

Console | Problems | Properties | Search | Tasks

```
<terminated> (exit value: 0) Prj_ClasseVeiculo_Cpp [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_ClasseVeiculo_Cpp/Debug/Prj_ClasseVeic
Marca: Toyota, Modelo: Corolla, Ano: 2022
```

5.2.3 Tratamento de exceções

O tratamento de exceções em C++ é uma técnica que permite lidar com situações anômalas ou erros durante a execução do programa.

Isso é feito para prevenir falhas inesperadas e permitir que o programa reaja a erros de uma maneira controlada. As principais palavras-chave envolvidas no tratamento de exceções são **try**, **catch**, e **throw**.



5.2.3 Tratamento de exceções

try: É usado para envolver o bloco de código no qual podem ocorrer exceções. O código dentro de um bloco try é monitorado para qualquer exceção que possa ser lançada.

catch: É usado para capturar e tratar exceções lançadas no bloco try. Cada bloco catch é associado a um tipo específico de exceção e é executado somente se uma exceção desse tipo for lançada.

throw: É usado para lançar explicitamente uma exceção. Pode ser usado para indicar que ocorreu uma condição excepcional.



5.2.3 Tratamento de exceções

O tratamento de erros e exceções desempenha um papel fundamental no desenvolvimento de software, contribuindo significativamente para a robustez, confiabilidade e manutenibilidade de um programa.

Vejamos algumas razões pelas quais o tratamento adequado de erros e exceções é crucial.



5.2.3 Tratamento de exceções

Prevenção de falhas Inesperadas:

O tratamento de exceções permite identificar e lidar com situações inesperadas que poderiam resultar em falhas no programa.

Em vez de um encerramento abrupto, o código pode capturar exceções e tomar ações apropriadas para manter o programa em um estado controlado.



5.2.3 Tratamento de exceções

Melhoria da resiliência:

Ao tratar exceções, um programa pode ser projetado para se recuperar de erros específicos e continuar a execução em um estado consistente.

Isso aumenta a resiliência do software, permitindo que ele continue funcionando mesmo quando ocorrem condições excepcionais.



5.2.3 Tratamento de exceções

Maior compreensão do código:

O tratamento de exceções ajuda a separar a lógica de tratamento de erros da lógica principal do programa.

Isso melhora a legibilidade do código, tornando mais fácil entender o fluxo principal da aplicação sem ser obscurecido por manipulação de erros.



5.2.3 Tratamento de exceções

Manutenção simplificada:

O tratamento adequado de exceções facilita a manutenção do código ao centralizar a lógica de tratamento de erros.

Isso significa que as alterações na maneira como os erros são tratados podem ser feitas em um único local, em vez de espalhadas por todo o código.



5.2.3 Tratamento de exceções

Feedback ao usuário:

O tratamento de exceções também oferece a oportunidade de fornecer feedback significativo ao usuário sobre o que deu errado e como corrigir o problema.

Isso contribui para uma melhor experiência do usuário e facilita a resolução de problemas.



5.2.3 Tratamento de exceções

Identificação antecipada de problemas:

Tratar exceções permite que os desenvolvedores identifiquem antecipadamente problemas potenciais durante o desenvolvimento e teste do software, melhorando a qualidade e confiabilidade do produto final.



5.2.3 Tratamento de exceções

Registro de erros:

Ao tratar exceções, os desenvolvedores têm a oportunidade de registrar informações detalhadas sobre o erro.

Esses registros podem ser valiosos para diagnóstico e correção de problemas em ambientes de produção.



5.2.3 Tratamento de exceções

O tratamento de erros e exceções é uma prática essencial para a construção de software robusto e confiável.

Ele contribui para a estabilidade do programa, facilita a manutenção e melhora a experiência do usuário ao lidar de maneira eficaz com situações excepcionais.

Vejamos como implementá-lo em C++.



5.2.3 Tratamento de exceções

```cpp
#include <iostream>
#include <stdexcept> // Necessário para std::runtime_error

int main() {
    try {
        // Código que pode gerar exceções
        int divisor = 0;
        if (divisor == 0) {
            throw std::runtime_error("Divisão por zero não permitida");
        }
        // O código abaixo não será executado se a exceção ocorrer acima
        int resultado = 10 / divisor;
        std::cout << "Resultado: " << resultado << std::endl;
    } catch (const std::runtime_error &e) {
        // Captura e trata exceções do tipo std::runtime_error
        std::cerr << "Erro: " << e.what() << std::endl;
    } catch (const std::exception &e) {
        // Captura e trata outras exceções derivadas de std::exception
        std::cerr << "Exceção: " << e.what() << std::endl;
    } catch (...) {
        // Captura qualquer outra exceção não tratada pelos blocos anteriores
        std::cerr << "Exceção desconhecida" << std::endl;
    }

    return 0;
}
```

Console × Problems Properties Search Tasks

<terminated> (exit value: 0) Prj_Excecoes [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_Excecoes/Debug/Prj_Excecoes (16/02/2024 20:25)

Erro: Divisão por zero não permitida

5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

Armazenamento de dados heterogêneos em memória secundária refere-se à prática de gravar informações de diferentes tipos e estruturas em um meio de armazenamento não volátil, como discos rígidos ou outros dispositivos de memória secundária.

Essa abordagem é utilizada quando é necessário preservar dados complexos e variados fora da execução do programa, possibilitando sua persistência para uso posterior.



5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

Para implementar o armazenamento de dados heterogêneos em memória secundária, são empregadas técnicas de serialização, que consistem na conversão de dados estruturados em uma representação linear, geralmente em forma de bytes.

Fluxo de dados: Objeto -> Bytes -> Arquivo (ou banco de dados)



5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

Essa prática é essencial para a persistência de informações complexas em aplicações que precisam armazenar configurações, estados de aplicativos, ou qualquer tipo de dados que deva ser mantido entre diferentes execuções do programa.

O armazenamento em memória secundária oferece flexibilidade e adaptabilidade, tornando-se uma técnica valiosa na construção de sistemas robustos e versáteis.



5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

Voltando para a Linguagem, C, vejamos a gravação em arquivo em duas versões diferentes:

I) Gravando e lendo um simples texto em um arquivo; e

II) Gravando e lendo uma struct com tipos de dados diferentes em um arquivo.



5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

Exemplo em C para gravação e leitura de um **texto** em um arquivo:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    // Abre o arquivo para escrita
    FILE *arquivo = fopen("exemplo.txt", "w");

    // Verifica se o arquivo foi aberto com sucesso para escrita
    if (arquivo != NULL) {
        // Escreve o texto no arquivo
        fprintf(arquivo, "Hello, World\n");

        // Fecha o arquivo de escrita
        fclose(arquivo);
        printf("Texto gravado com sucesso.\n");
    } else {
        // Se houver erro na abertura do arquivo para escrita
        printf("Erro ao abrir o arquivo para escrita.\n");
        return 1; // Retorna um código de erro
    }
```



5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

Exemplo em C para gravação e leitura de um **texto** em um arquivo:

```c
    // Abre o arquivo para leitura
    arquivo = fopen("exemplo.txt", "r");

    // Verifica se o arquivo foi aberto com sucesso para leitura
    if (arquivo != NULL) {
        char linha[100]; // Define um buffer para armazenar a linha lida

        // Lê e imprime cada linha do arquivo
        while (fgets(linha, sizeof(linha), arquivo) != NULL) {
            printf("Conteúdo lido do arquivo: %s", linha);
        }

        // Fecha o arquivo de leitura
        fclose(arquivo);
    } else {
        // Se houver erro na abertura do arquivo para leitura
        printf("Erro ao abrir o arquivo para leitura.\n");
        return 1; // Retorna um código de erro
    }

    return 0;
}
```

Console | Problems | Properties | Search | Tasks

```
<terminated> (exit value: 0) Prj_File [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_File/Debug/Prj_File (16/02/2024 21:01)
Texto gravado com sucesso.
Conteúdo lido do arquivo: Hello, World
```

5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

FILE: FILE é um tipo de dados em C que representa um arquivo. Variáveis desse tipo são usadas para realizar operações de leitura e escrita em arquivos.

fopen: fopen é uma função que abre um arquivo. Neste caso, fopen("exemplo.txt", "w") abre um arquivo chamado "exemplo.txt" para escrita ("w" significa write). Se o arquivo não existir, ele será criado. Se já existir, seu conteúdo será truncado.

5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

fprintf: fprintf é uma função que funciona de maneira semelhante ao printf, mas escreve os dados no arquivo especificado. Neste exemplo, fprintf(arquivo, "Hello, World\n"); escreve a string "Hello, World" no arquivo.

fclose: fclose é usado para fechar o arquivo que foi aberto anteriormente. Neste caso, fclose(arquivo); fecha o arquivo após a escrita.

5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

fgets: fgets é uma função que lê uma linha do arquivo. Neste caso, é usada em um loop para ler cada linha do arquivo e imprimir na tela. A função fgets(linha, sizeof(linha), arquivo) lê uma linha do arquivo apontado por arquivo para o array linha com um tamanho máximo de sizeof(linha) caracteres.

O programa começa abrindo um arquivo chamado "exemplo.txt" para escrita, escreve "Hello, World" nele, fecha o arquivo e depois o abre para leitura. Ele lê cada linha do arquivo e imprime o conteúdo na tela.

Exemplo em C para gravação e leitura de uma **struct** em um arquivo:

```c
#include <stdio.h>
#include <stdlib.h>

// Definição da estrutura
struct ExemploStruct {
    int numero;
    float decimal;
    char texto[50];
};

int main(void) {
    // Abre o arquivo para escrita binária
    FILE *arquivo = fopen("exemplo_struct.bin", "wb");

    // Verifica se o arquivo foi aberto com sucesso
    if (arquivo != NULL) {
        // Cria uma instância da estrutura
        struct ExemploStruct dadosParaGravar = { 13, 3.14,
                "Estrutura de exemplo" };

        // Escreve a estrutura no arquivo
        fwrite(&dadosParaGravar, sizeof(struct ExemploStruct), 1, arquivo);

        // Fecha o arquivo
        fclose(arquivo);
        printf("Estrutura gravada com sucesso.\n");
    } else {
        // Se houver erro na abertura do arquivo
        printf("Erro ao abrir o arquivo para escrita.\n");
    }
```



Exemplo em C para gravação e leitura de uma **struct** em um arquivo:

```c
    // Abre o arquivo para leitura binária
    FILE *arquivoLeitura = fopen("exemplo_struct.bin", "rb");

    // Verifica se o arquivo foi aberto com sucesso
    if (arquivoLeitura != NULL) {
        // Cria uma instância da estrutura para armazenar os dados lidos
        struct ExemploStruct dadosLidos;

        // Lê a estrutura do arquivo
        fread(&dadosLidos, sizeof(struct ExemploStruct), 1, arquivoLeitura);

        // Fecha o arquivo de leitura
        fclose(arquivoLeitura);

        // Exibe os dados lidos
        printf("Dados lidos da estrutura: Numero: %d, Decimal: %.2f, Texto: %s\n",
               dadosLidos.numero, dadosLidos.decimal, dadosLidos.texto);
    } else {
        // Se houver erro na abertura do arquivo de leitura
        printf("Erro ao abrir o arquivo para leitura.\n");
    }

    return 0;
}
```

Console | Problems | Properties | Search | Tasks

```
<terminated> (exit value: 0) Prj_File_Struct [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_File_Struct/Debug/Prj_File_Struct (16/02/2024 21:16)
Estrutura gravada com sucesso.
Dados lidos da estrutura: Numero: 13, Decimal: 3.14, Texto: Estrutura de exemplo
```

5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

fwrite (file write) é uma função em C usada para escrever dados em um arquivo. Ela recebe quatro parâmetros:

- Um ponteiro para os dados que você deseja escrever (ptr).
- O tamanho em bytes de cada elemento a ser escrito (size).
- O número de elementos a serem escritos (count).
- Um ponteiro para o objeto FILE que representa o arquivo (stream).

A função grava os dados no arquivo de acordo com os parâmetros fornecidos.

5.3 Noções sobre armazenamento de dados heterogêneos em memória secundária

fread (file read) é uma função em C usada para ler dados de um arquivo.

Ela também recebe quatro parâmetros:

- Um ponteiro para o buffer onde os dados lidos serão armazenados (ptr).
- O tamanho em bytes de cada elemento a ser lido (size).
- O número de elementos a serem lidos (count).
- Um ponteiro para o objeto FILE que representa o arquivo (stream).

A função lê os dados do arquivo de acordo com os parâmetros fornecidos e os armazena no buffer especificado.

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:

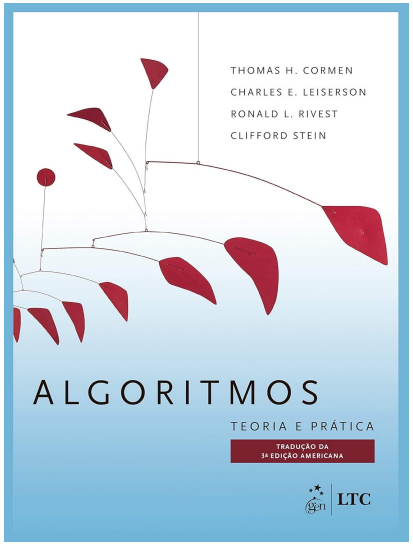
- CORMEN, Thomas. **Algoritmos** - Teoria e Prática. Editora GEN LTC. 3ª edição, 2012.

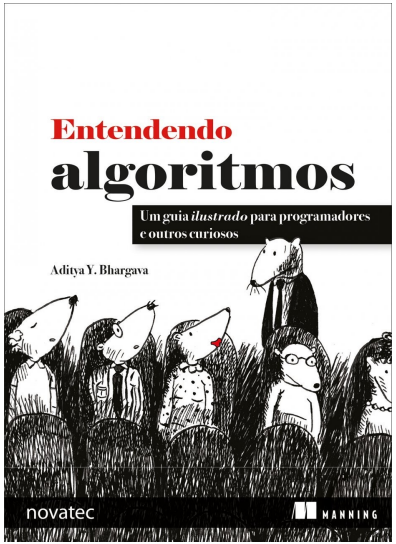
- BHARGAVA, Aditya Y. **Entendendo Algoritmos**: Um Guia Ilustrado Para Programadores e Outros Curiosos. Novatec Editora, 2017.

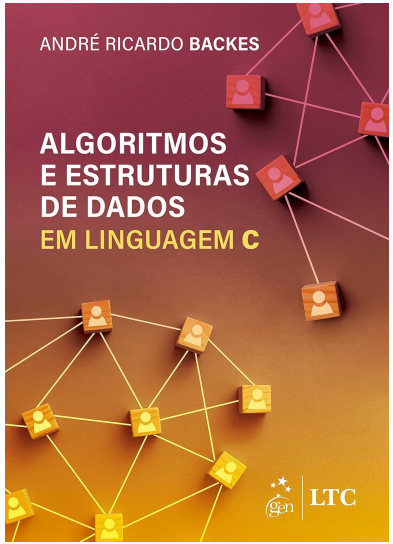
- BACKES, André Ricardo. **Algoritmos e Estruturas de Dados em Linguagem C**. Editora LTC, 2022.

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:







Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:

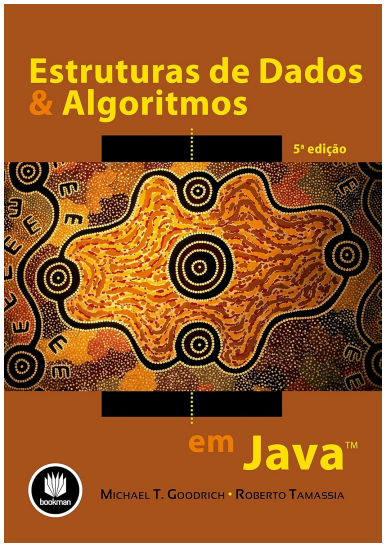
- GOODRICH, Michael T. **Estruturas de Dados e Algoritmos em Java**. Editora Bookman. 5ª edição, 2013.

- AGARWAL, Basant. **Estruturas de Dados e Algoritmos com Python**: Armazene, manipule e acesse dados de forma eficaz e melhore o desempenho de suas aplicações. Novatec Editora, 2023.

- GRONER, Loiane. **Estruturas de Dados e Algoritmos com JavaScript**: Escreva um Código JavaScript Complexo e Eficaz Usando a Mais Recente ECMAScript. Novatec Editora, 2019.

Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:







Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:

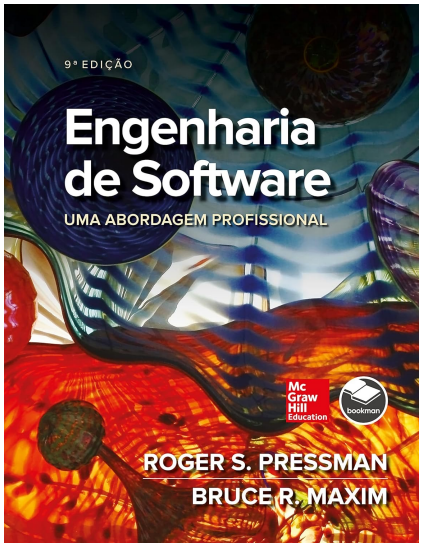
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**. Editora Grupo AMGH, 2021.

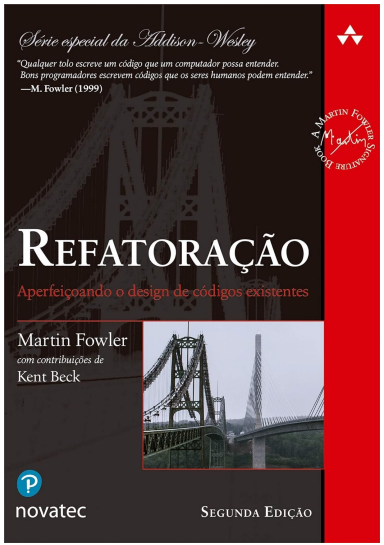
- FOWLER, Martin. **Refatoração**: Aperfeiçoando o Design de Códigos Existentes. Editora Novatec, 2020.

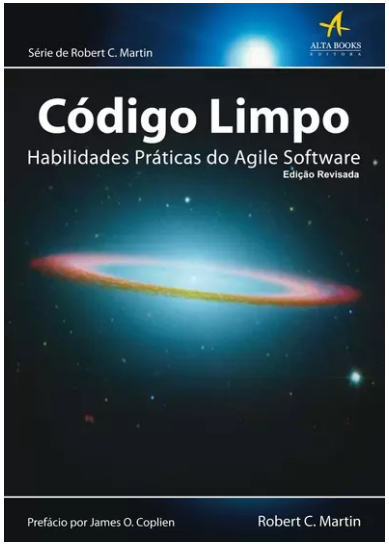
- MARTIN, Robert C. **Código limpo**: habilidades práticas do Agile software. Editora Alta Books, 2009.

Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:







Disciplina – Algoritmos e Estruturas de Dados I

Nesta disciplina usaremos: Linguagem C (ANSI Style)

- Linguagem C
[DevDocs] https://devdocs.io/c/

- Eclipse IDE
[Download] https://www.eclipse.org/downloads/packages/release/2023-12/r/eclipse-ide-cc-developers
[Docs] https://www.eclipse.org/documentation/





Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com





GitHub LinkedIn Lattes